

ECE 310 – Fall 2025

Project 3

Summary

In this project, you will be designing and implementing an Arithmetic Logic Unit (ALU) to perform addition and subtraction of Binary Coded Decimal (BCD) numbers using serial communication.

1. Introduction

The ALU will add or subtract 4-digit BCD numbers (16 bits each), producing a 5-digit BCD (20-bit) result. The BCD values and control for add/sub operations are provided to the system serially and are output from the system serially (data will be provided as a “packet”). Operations inside the ALU are performed on parallel data, so the system requires a serial input parallel output (SIPO) register on the input and parallel input serial output (PISO) register on the output.

1.1 Binary Coded Decimal (BCD)

Binary Coded Decimal (8-4-2-1 code, 8421 code, or simply BCD) is a weighted code for representing decimal digits as binary data. Since the range of decimal numbers is 0 to 9, only 4 bits are required for encoding [3:0], where each bit is weighted according to the 8421 encoding:

$$8 * bit[3] + 4 * bit[2] + 2 * bit[1] + 1 * bit[0]$$

Since only 10 combinations out of the 4-bit values are used, the remaining 6 combinations are not used and considered invalid for representing BCD values, so they will not appear as inputs in this project.

1.2 BCD Addition

BCD addition works in the same way that binary addition works. The difference is when the addition produces a result in BCD that is larger than 9, an adjustment is needed – subtract 10 from the result and provide a carry in the next digit position.

For example:

$$\begin{aligned} 6_{10} + 7_{10} &= 13_{10} \\ 0110_2 + 0111_2 &= 1101_2 \\ 0110_{BCD} + 0111_{BCD} &= 1101_2 \\ &= 0001 (1101_2 - 1010_2) \\ &= 0001 0011_{BCD} \end{aligned}$$

Subtracting 10_{10} can be done via addition of -10_{10} , or adding 0110_2 in binary. This means that the result can be correct in BCD if either 10_{10} is subtracted, or, even better, by adding 6_{10} , which also generates the carry that allows to create the next BCD digit correctly.

For example:

$$\begin{aligned}
 6_{10} + 7_{10} &= 13_{10} \\
 0110_2 + 0111_2 &= 1101_2 \\
 0110_{BCD} + 0111_{BCD} &= 1101_2 > 9_{10} \\
 &= 1101_2 + 0110_2 \\
 &= 10011_2 \\
 &= 0001\ 0011_{BCD}
 \end{aligned}$$

1.3 BCD Subtraction

BCD subtraction is more involved than binary subtraction, it's not as simple as taking the 2's complement and adding. The reason is the four bits are encoding a base-10 value and the encoding is not self-complementing, i.e. we need 10's complement. Similar to 2's complement in binary, any base's complement can be obtained by subtracting each digit from the (base-1) value and then adding 1 back as the LSB. For base-10, each digit must be subtracted from 9 to get 9's complement and then add 1 as the LSB.

For example:

$$\begin{aligned}
 0011\ 1001\ 1000\ 0000_{BCD} &= 3980_{10} \\
 0110\ 0000\ 0001\ 1001_{BCD} + 0001_{BCD} &= 6019_{10} + 1_{10} \\
 0110\ 0000\ 0010\ 0000_{BCD} &= 6020_{10}
 \end{aligned}$$

In the last addition, the LSB added a value of 6_{10} in order to generate a carry for the next digit and because adding 1_{10} exceeded the range of BCD numbers. According to the result above, 6020_{10} is the 10's complement of 3980_{10} . This should make sense logically as adding $A + (-A) = 0$ for any number, which is also the case for the number above as well.

A BCD subtraction example:

$$\begin{aligned}
 637_{10} - 459_{10} &= 637_{10} + (540_{10} + 1) \\
 &= 637_{10} + 541_{10} \\
 &= 1\ 178_{10}
 \end{aligned}$$

Similar to binary subtraction, the borrow (carry in the next digit position) is discarded and the result becomes 178_{10} .

2. Serial BCD ALU

2.1 Operation

An active-high reset signal (asserted once for the entire simulation) will start the operation of feeding a bit stream of data to be operated on input **din**. There is no control signal that will indicate the start of a “packet”, instead a particular control pattern will be used to identify the start of the “packet”. The control pattern is **8'b01011010 (8'h5A)**, after which the next bit will denote the operation to be performed: **1'b0** indicates an addition, **1'b1** indicates subtraction operation. Then two 16-bit values will be provided, each representing a 4-digit BCD value, first number A, then number B.

For example, if the module needs to compute the operation $3627 + 1287$, it would send the following stream of serial data **MSB first**. Underscores are valid Verilog and are used to provide an easier way to read the bitstream but will not be provided as part of actual input:

01011010_0_0011_0110_0010_0111_0001_0010_1000_0111

Once the module has computed the result, it will shift data serially out of the single bit output named **result** MSB first. Similar to the input “packet”, the output will be constructed as a packet with a control pattern of **8'b10010110 (8'96h)** following by the 5-digit BCD result (20 bits). Given the example above, the resulting output stream would be (4914):

10010110_0000_0100_1001_0001_0100

2.2 Sequence Detection

Detecting the input is a matter of sequence detection, so data must be continuously shifted into a 41-bit register (8-bit pattern, 1-bit op, 2× 16-bit operands) or another circuitry that will detect the header. As the data shifts in, the control bits will be used to indicate that the register contains all the information for the operation. At this point, the module should be processing the data to perform the operation and must clear the register so that data in A or B is not mistaken for the control sequence, so a non-overlapping sequence must be detected.

2.3 Timing

The output stream will have 28 bits (8-bit pattern, 20-bit data), so it is possible for the output stream to be completely independent timing-wise from the input (41 bits). This means that it is possible for the input stream to be continuously fed to the module without any interruptions to wait for the current operation to complete because it takes longer to shift in new operation bits than output the result of the current operation.

2.4 Requirements

The Serial BCD ALU shall conform to the following requirements.

- **Naming Requirements:**

- Top-level module shall be called **Project3**.
- There shall be an active-High synchronous reset signal named **reset** that will reset the entire system on the next rising edge when asserted. The signal can be asserted at any point during operation.
- There shall be a free-running clock named **clock**.

- There shall be a 1-bit input named **din** and a 1-bit serial output named **result**.
- **Timing Requirements**
 - The reset signal will initialize the system to a known state.
 - The ALU shall be able to continuously perform operations one after another without a break.
- **Implementation Requirements**
 - Free to choose any level of modeling for this ALU or a mix of models.

Submission Details:

- Submit 1 .v file with your model to Moodle. A testbench is not required.
- Submit 1 .pdf file with a clear waveform running through a few examples of inputs. **Ensure the waveform has enough resolution to be visible for your test inputs and outputs, it can be broken down into multiple figures if needed to get proper resolution.** You can also include a short description of code, approach, code snippets, etc, in case there are issues with compiling your code. It may help to perform the operations on paper so that the work may serve as an explanation for the results seen in the waveforms.
- In the .pdf file, also include a drawing of the architecture of your module so that your approach to the design can be inspected. Add the synthesis results from your Vivado project and compare and contrast it with your pen-and-paper/digital implementation.

Grading breakdown:

3 pts – Model (.v) file upload

2 pts – Report (.pdf) upload

15 pts – Diagram of pen-and-paper/digital architecture and synthesis results. Include a discussion comparing and contrasting the two diagrams. If synthesis was not successful, include a detailed description of issues encountered, solutions tried, and an expectation of what your diagram would be synthesized as (remember that some constructs map differently from code to hardware during synthesis -- see Synthesis slides for more).

10 pts – Clear waveform of running through a few examples of inputs. Make sure to cover as many test cases as reasonably possible and describe the list of tests.

5 pts – Discuss your approach to coding this project. Include any discussion on your code structure (especially if using nested modules), additional explanations on code, approach, code snippets, software architecture, etc.

5 pts – Asserting the reset signal will put the system in a known state (generally all zeros).

5 pts – Correctly detecting the start of packet (0x5A).

2 pts – Correctly identifying the operation from the input stream (0 for add, 1 for sub).

8 pts – Correctly identifying 1st and 2nd operands from the input stream (data deserialization).

5 pts – Correctly performing the addition operation.

10 pts – Correctly performing the subtraction operation. Includes performing 9's complement and ignoring the most significant digit(s) as needed.

2 pts – Outputting the correct output header (0x96).

4 pts – Proper serialization of outputs. Most significant digit first, etc.

4 pts – Proper timing on inputs (41 cycles) and outputs (28 cycles).

5 pts – Continuous operation (inputs are able to be fed to the system without any breaks).

5 pts – Operation with padding between valid input streams (may have a variable number of cycles of break where the header is not present).

10 pts – Passing tests. A list is not provided, but consider the values for possible inputs that need to present correct outputs (e.g. all zeros, etc.)

Extra Credit – up to 10 pts for writing report above using Latex and uploaded as a .pdf .